Atomic-to-Compositional Generalization for Mobile Agents with A New Benchmark and Scheduling System

Yuan Guo, Tingjia Miao, Zheng Wu, Pengzhou Cheng, Ming Zhou, Zhuosheng Zhang

Paper version: arXiv 2506.08972v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *prior work has predominantly focused on atomic tasks… while overlooking … compositional tasks*

Real asks span apps and branch on retrieved info; atomic executors stall — under-execution, attention drift.

Why compositional tasks break agents

1. **Long horizon.** Subtasks dropped: attention drift.
2. **Context transfer.** Info lost between apps.
3. **Think vs act.** Tapping where reasoning is needed.

Evaluation setup

UI-NEXUS · 100 templates | 20 local + 30 online apps | avg 14.05 optimal steps | GPT-4o backbones | Qwen2-VL-7B agents | budget ≈ 2× optimal

- **ANCHOR:** emulator, automatic rewards. Online apps: judged.
- **Baselines:** 3 GPT-4o workflows + OS-Atlas / UI-TARS 7B.

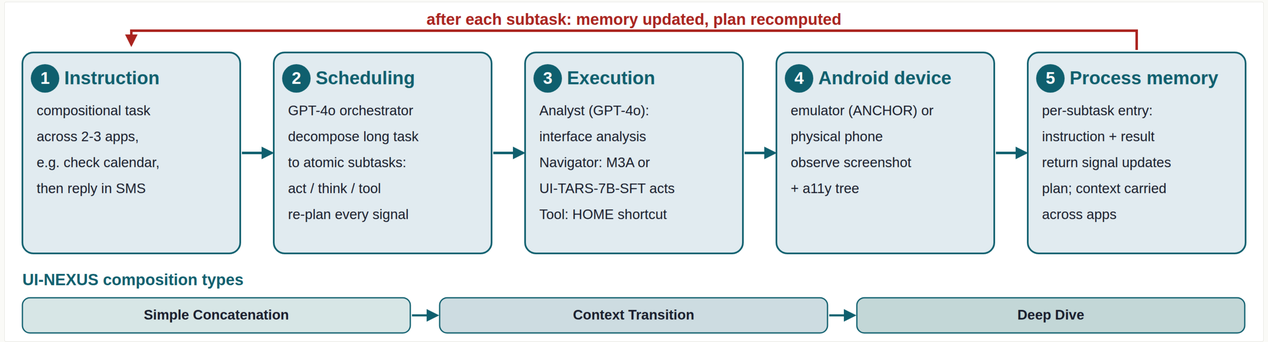
Our read (CUA Radar)

61	★★	online
score of 100	band	evidence level

- **Reports:** online eval; strong baselines; cost; gap study; code.
- **Flags:** no seeds/variance; held-out env unclear; judge untested.
- Pattern C17. Authors' numbers, not rerun.

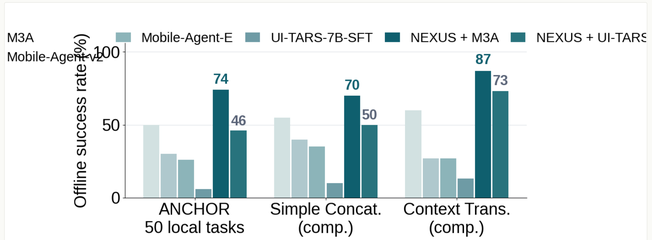
AGENT-NEXUS: schedule, execute, remember

- **Sees:** screenshots + a11y tree on Android; **acts:** tap / type / swipe via the navigator agent.
- **Scheduler:** GPT-4o orchestrator decomposes the task into atomic subtasks typed act / think / tool.
- **Signal:** each subtask's return signal feeds process memory; plan recomputed: $\pi_{t+1} = M(s_{t+1}, g, P_{t+1})$ (Eq. 9).



Our redraw (not the paper's figure): the AGENT-NEXUS scheduling loop with process memory.

Results: success rate (%)



Tables 2 + 5, redrawn — all OFFLINE: ANCHOR emulator tasks, automatic reward signals.

Reported: 24% to 40% success-rate improvement on compositional tasks

SO WHAT → *Online apps (En / Cn): NEXUS+UI-TARS 28.0 / 32.0 — Controls table.*

Atomic vs compositional

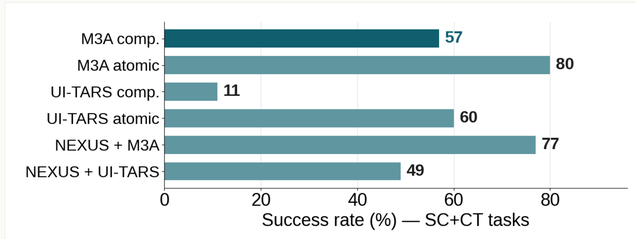


Table 5, redrawn: success with the direct compositional instruction vs manual atomic decomposition (strong ceiling).

SO WHAT → *Atomic instructions lift M3A 57→80 (↑87%) and UI-TARS 11→60 (↑452%); AGENT-NEXUS recovers PGR 88 / 76 without a human decomposer. OS-Atlas stays near zero (3.0) even decomposed. Wrapped agents (77.0 / 49.0) clear every baseline's compositional row (max 57.0): the scheduler hands each executor one self-contained atomic instruction per global step.*

Headline Numbers

74.0%
ANCHOR success
M3A alone: 50.0

46.0%	UI-TARS wrapped
24-40%	reported gain
88 / 76	PGR M3A / UI-TARS

Controls

CONTROL	RESULT	READING
Online En/Cn	NEXUS 28/32	M3A 32/4
Cost/step	0.040 vs 0.037	vs M3A 0.037
Latency/step	18.86 vs 14.77	vs M3A 14.77
Step budget	2× optimum	all agents
Online judge	MLLM + human	unvalidated

Takeaway

A GPT-4o scheduler makes atomic executors compositional: reported gain 24% to 40% on UI-NEXUS, at $0.040 per step with the M3A navigator. Process memory carries context across apps.

100 templates · 50 apps | avg 14.05 optimal steps | plug-in navigator | PGR 88 / 76 | gain 24% to 40%

Scan to Read

Paper | Project